

Puppy Image Multi-Scale Representation in OpenCV

John Wensink

CSC515 Foundations of Computer Vision

Colorado State University – Global Campus

Dr. Bingdong Li

March, 30, 2025

Puppy Image Multi-Scale Representation in OpenCV

This week, we began to explore some simple pixel matrix transformations using Python and OpenCV. The idea that is central to all of these transformations is that images can be described using a three-dimensional pixel matrix, with each pixel storing values for red, green, and blue intensity channels. These matrices form the foundation for visual data processing and computer vision. We applied this theoretical understanding in an interactive assignment involving an image of a puppy. The first step involved importing the image from the file path.

```
1  import cv2
2
3  #Load the image from the local file path
4  image_path = r"C:\Users\jwens\Desktop\CSUGlobal\CSC515\Image_Puppy.jpg"
5  image = cv2.imread(image_path)
```

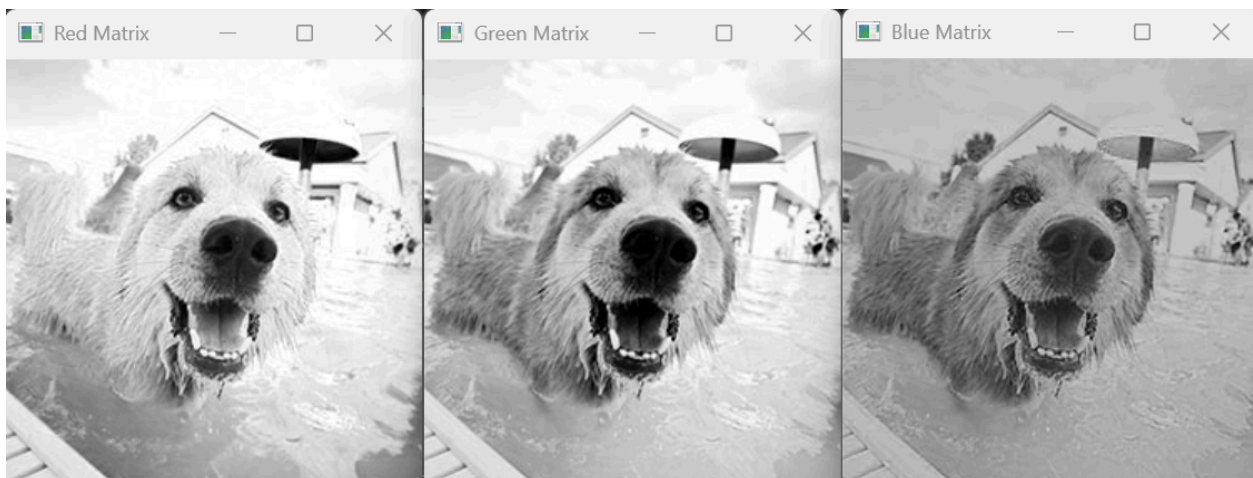
After successfully importing the image, the next step is to extract each individual color channel from the image. Since the original image is a 3D matrix representing RGB channels, the function *cv2.split()* (Devara, 2023) is used to separate these into three distinct 2D matrices. Each resulting matrix represents the intensity of one color across all pixels in the image. Interestingly enough, the code *red, green, blue = cv2.split(image)* is incorrect. OpenCV does not order the channels as RGB by default, rather, it uses the order BGR. This nuance in OpenCV's design caused issues when I assumed that the channels were arranged in the traditional RGB format. As a result, the outputs of *cv2.split(image)* led to mismatches between the intended and the actual color data.

The correct code looks as follows.

```
7     #Extract color channels as blue, green, red
8     blue, green, red = cv2.split(image)
9     red_matrix = red
10    green_matrix = green
11    blue_matrix = blue
```

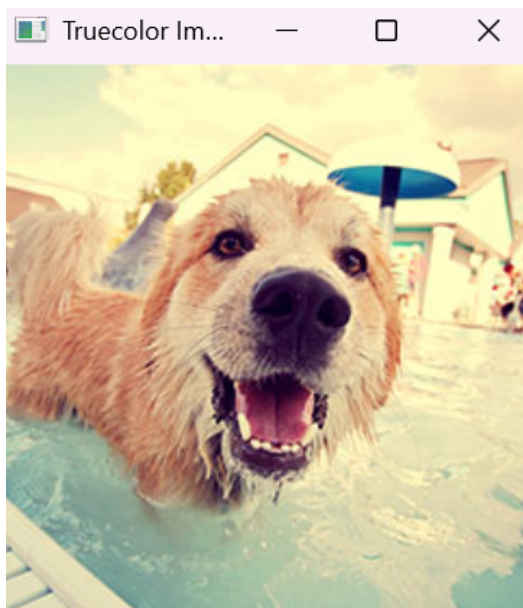
With the channels now split, we are ready to inspect each individual color by displaying it as a greyscale image of that channel. The `cv2.imshow()` function will need to be called three times, once for each channel. Each call creates a window showing the respective channel matrix, allowing the user to observe the distribution of that color's intensity across the image. Each pixel's brightness in these displays represents the strength of that particular color at that pixel's location. This technique is useful for analyzing how individual channels contribute to the full image and is particularly helpful for filtering and preprocessing in computer vision pipelines.

```
13    #Display each color channel as grayscale
14    cv2.imshow("Red Channel", red_matrix)
15    cv2.imshow("Green Channel", green_matrix)
16    cv2.imshow("Blue Channel", blue_matrix)
```



After separating and analyzing the individual color channels, the matrices can be recombined using OpenCV's `cv2.merge()` function (Devara, 2023.) The resulting output gets stored in a variable named `truecolor.image`.

```
18 #Merge channels back into the original truecolor image
19 truecolor_image = cv2.merge((blue_matrix, green_matrix, red_matrix))
20 cv2.imshow("Truecolor Image (Original RGB)", truecolor_image)
```



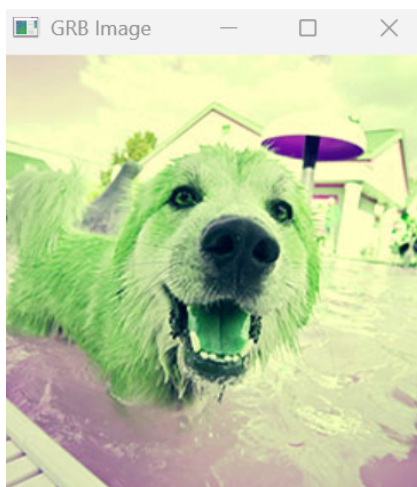
Displaying this merged image validates that the individual channel matrices have been correctly aligned and reassigned, confirming the integrity of the channel-splitting and merging process.

The next requirement asks us to imagine what the image would look like if we swapped the reds with the greens. Thankfully, we've already discovered that the BGR order matters in OpenCV, and I got a hint at what a swapped channel would look like when I accidentally followed the RGB standard at first. If we swapped the red and the green channels, we would expect (based on the intensity map when we split the channels) to see visual differences in the areas where reds and greens were most prominent. Specifically, regions that originally appeared green would take

on reddish hues, and those that were red would take on greenish hues. This is because we're not simply adjusting color tone, we're fundamentally reassigning which color each pixel's intensity contributes to. Looking at the greyscale images side by side, we can anticipate how much this transformation will affect the final image. The red matrix shows where high-intensity values are present, particularly around the sunlit portions of the dog's fur. Similarly, the green matrix is brightest around the mid-tones of the dog's fur, where green is more dominant. When these two channels get swapped, the color mapping should be reversed. To accomplish this transformation, all we have to do is swap the order in which the channels appear in the code. Again, using `cv2.merge()`, the final color output is based on the sequence in which the channels are passed. By changing the order from BGR to BRG, we've effectively reassigned each pixel's red intensity to green and vice versa.

```
22 #Swap Red and Green Channels to GRB
23 GRB_image = cv2.merge((blue_matrix, red_matrix, green_matrix))
24 cv2.imshow("GRB Image", GRB_image)
```

The result is a GRB image with distorted color mappings, showing an unusual blend of greens and purples that originally had more natural red and green tones.



As usual with OpenCV, it is useful to include a final block of code that keeps the program's output windows open until the user decides to close them. The function `cv2.waitKey(0)` (OpenCV, n.d.) waits until any key is pressed with the output windows open before commanding the windows to close. Without such code, the windows would close almost instantly. Alternatively, the argument `(0)` could be replaced with a time in milliseconds to wait before the window is closed (GeeksforGeeks, n.d.) Here, we will use `(0)` to facilitate taking screenshots. Once a key is pressed, `cv2.destroyAllWindows()` is called to cleanly close all image display windows created by OpenCV during the program's execution. This cleanup step is good practice to release any resources that are being held by the graphical interface and ensures a clean exit from the program.

```
26     #Wait for key press and clean up
27     cv2.waitKey(0)
28     cv2.destroyAllWindows()
```

References

Devara, G. (2023, May 30). Splitting and merging channels with OpenCV - Python.

Tutorialspoint. Retrieved March 30, 2025, from:

<https://www.tutorialspoint.com/splitting-and-merging-channels-with-opencv-python>

GeeksforGeeks. (2021, October 7). Python OpenCV – waitKey() function. GeeksforGeeks.

Retrieved March 30, 2025, from

<https://www.geeksforgeeks.org/python-opencv-waitkey-function/>

OpenCV. (n.d.). High-level GUI (highgui) — OpenCV documentation. OpenCV. Retrieved

March 30, 2025, from https://docs.opencv.org/4.x/d7/dfc/group_highgui.html